

React

useReducer

ReactにはComponentの状態を管理するuseStateがある。

しかし、ビジネスロジックが複雑になるとuseStateを使用することは難しい。

そのため、useReducerを使う。

useStateは銀行の帳簿に顧客が直接アクセスすること

useReducerは顧客が職員に注文することである。

useReducerは状態の変更を隠すことができるし、顧客は複雑なロジックをきにする
ことも必要ない。

```
const [state, dispatch] = useReducer(reducer, initialState);
```

state : componentで使用する現在の値。

dispatch : 状態を変更することを要求する関数。

reducer : 関数をreturnすること。

initialState : 状態の初期値。

```
import React, { useReducer } from 'react';

function countReducer(oldCount, action) {
  if (action === 'UP') {
    return oldCount + 1;
  } else if (action === 'DOWN') {
    return oldCount - 1;
  } else if (action === 'RESET') {
    return 0;
  }
}

export default function Counter() {
  const [count, countDispatch] = useReducer(countReducer, 0);

  return (
    <div>
      <button onClick={() => countDispatch('DOWN')}>-</button>
      <button onClick={() => countDispatch('RESET')}>0</button>
      <button onClick={() => countDispatch('UP')}>+</button>
      <span>{count}</span>
    </div>
  );
}
```

- 0 + 1

Redux

```

import React, { useState } from 'react';
import './style.css';

export default function App() {
  const [number, setNumber] = useState(1);
  return (
    <div id="container">
      <h1>Root : {number}</h1>
      <div id="grid">
        <Left1></Left1>
        <Right1></Right1>
      </div>
    </div>
  );
}

function Left1(props) {
  return (
    <div>
      <h1>Left1 </h1>
      <Left2></Left2>
    </div>
  );
}

function Left2(props) {
  return (
    <div>
      <h1>Left2 : </h1>
      <Left3></Left3>
    </div>
  );
}

```

```

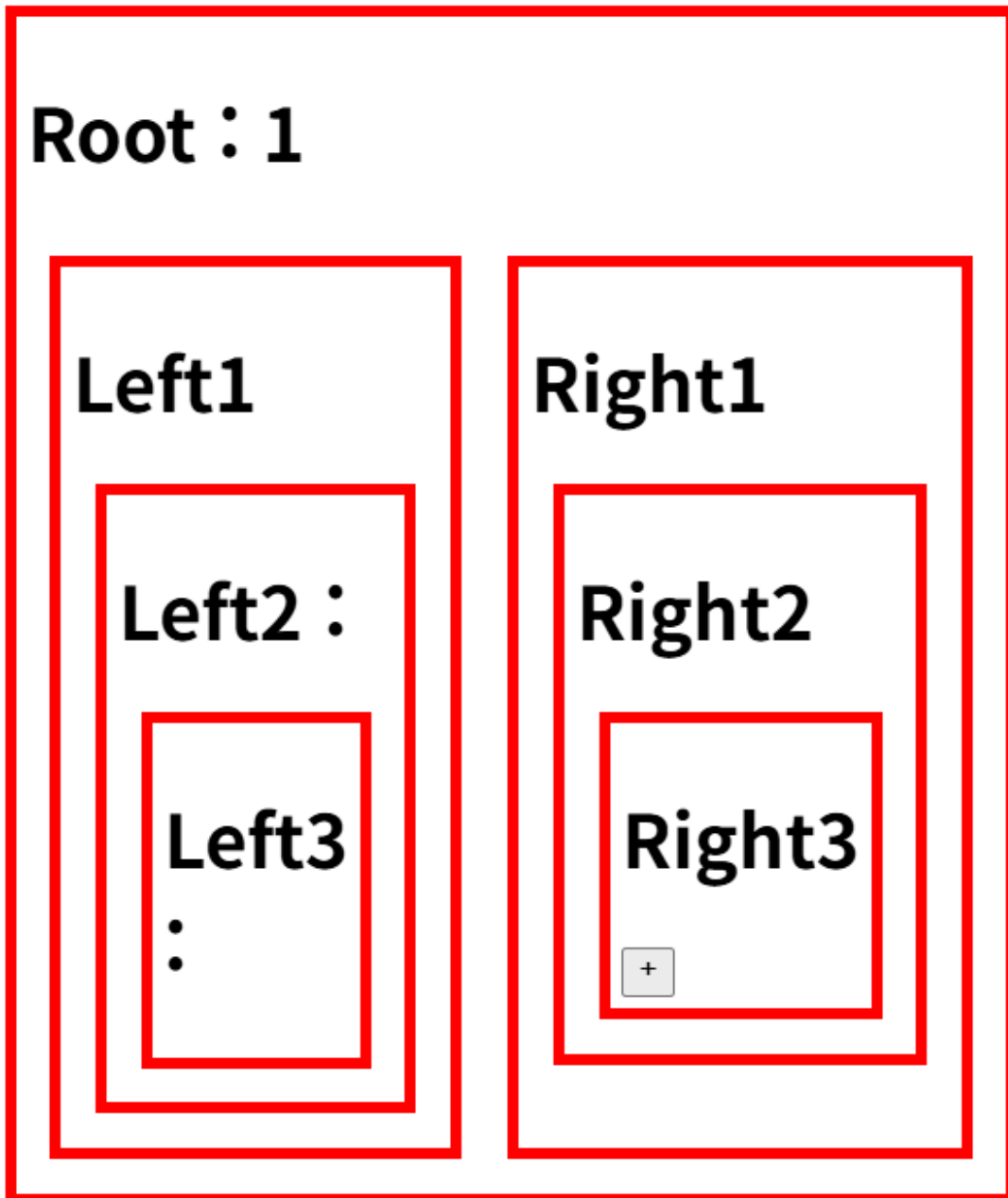
function Left3(props) {
  return (
    <div>
      <h1>Left3 : </h1>
    </div>
  );
}

function Right1(props) {
  return (
    <div>
      <h1>Right1</h1>
      <Right2></Right2>
    </div>
  );
}

function Right2(props) {
  return (
    <div>
      <h1>Right2</h1>
      <Right3></Right3>
    </div>
  );
}

function Right3(props) {
  return (
    <div>
      <h1>Right3</h1>
      <input type="button" value="+" onClick={() => {}}></input>
    </div>
  );
}

```



まず、このコードをみると、Rootとbuttonの間のcomponentが4つじゃなくて大量なら

全部つながるべきなので複雑なことになる。

そのため、Reduxを使う。

コードをみると、setNumber関数を伝えるためにRight1→2→3、Left1→2→3になる。

Reduxはこの問題を解決するため、componentが直接使えるようにする。

```
function reducer(currentState, action) {
  if (currentState === undefined) {
    return {
      number: 1,
    };
  }
  const newState = { ...currentState };
  if (action.type === 'PLUS') {
    newState.number++;
  }
  return newState;
}
```

```
const store = createStore(reducer);
```

reducerを定義して状態がどのようなになるかを定義する。

```
<div id="grid">
  <Provider store={store}>
    <Left1></Left1>
    <Right1></Right1>
  </Provider>
</div>
```

Providerを使ってLeft, Rightをstoreに接近することができるようにする。

```
function Left3(props) {
  console.log('3');
  const number = useSelector((state) => state.number);
  return (
    <div>
      <h1>Left3 : {number}</h1>
    </div>
  );
}
```

useSelectorを使ってstate.numberを取得する。

```
function Right3(props) {
  const dispatch = useDispatch();
  return (
    <div>
      <h1>Right3</h1>
      <input
        type="button"
        value="+"
        onClick={() => {
          dispatch({ type: 'PLUS' });
        }}
      ></input>
    </div>
  );
}
```

そして、useDispatchをすればPLUSができる。

